# GetRescued Application Documentation

Roadside assistance and towing CRM built on Zoho Creator. Developer reference, functionality guide and handover document.

Oct 12, 2026 · @Subash

## Project overview

GetRescued is a roadside-assistance and towing CRM built on Zoho Creator. It carries a breakdown from the moment the customer's call rings to the moment the vendor is paid, across two service lines that share one ticket but branch at almost every step.

| | RSR | TOW |
|---|---|---|
| Meaning | On-site repair | Recovery to a drop location |
| Field party | Vendor, then a technician | Vendor, then a driver |
| Locations needed | Breakdown only | Breakdown **and** drop |
| Search radius | 8 km | 20 km |
| Ends at | Issue resolved on the spot | Vehicle handed over at the drop point |

The branch is driven by one field, `Service_Type` on `Create_Case`, holding `RSR` or `TOW`.

### Who uses it

| Role | Where they work | What they do |
|---|---|---|
| Call-centre agent | callCenter widget | Answers the inbound call, searches for an existing case or starts a new one |
| Agent | getRescueTicket widget | Runs the ticket through the wizard: customer, fee, location, assignment, closure |
| Ops manager | getRescueTicket (elevated) | Approves vendor payouts, handles rejections and exceptions |
| Vendor | vendorTicket widget | Accepts or rejects the job, dispatches a technician or driver, closes the case |
| Technician | technicianTicket widget | Runs an RSR job on the ground |

| Role | Where they work | What they do |
|---|---|---|
| Driver | driverTicket widget | Runs a TOW job: pickup, transit, unloading, handover |
| Customer | WhatsApp only | Receives updates, shares a location pin, pays |

The customer has no login. Everything they see arrives as a WhatsApp message, and everything they send back comes through one inbound webhook.

## Scope of this guide

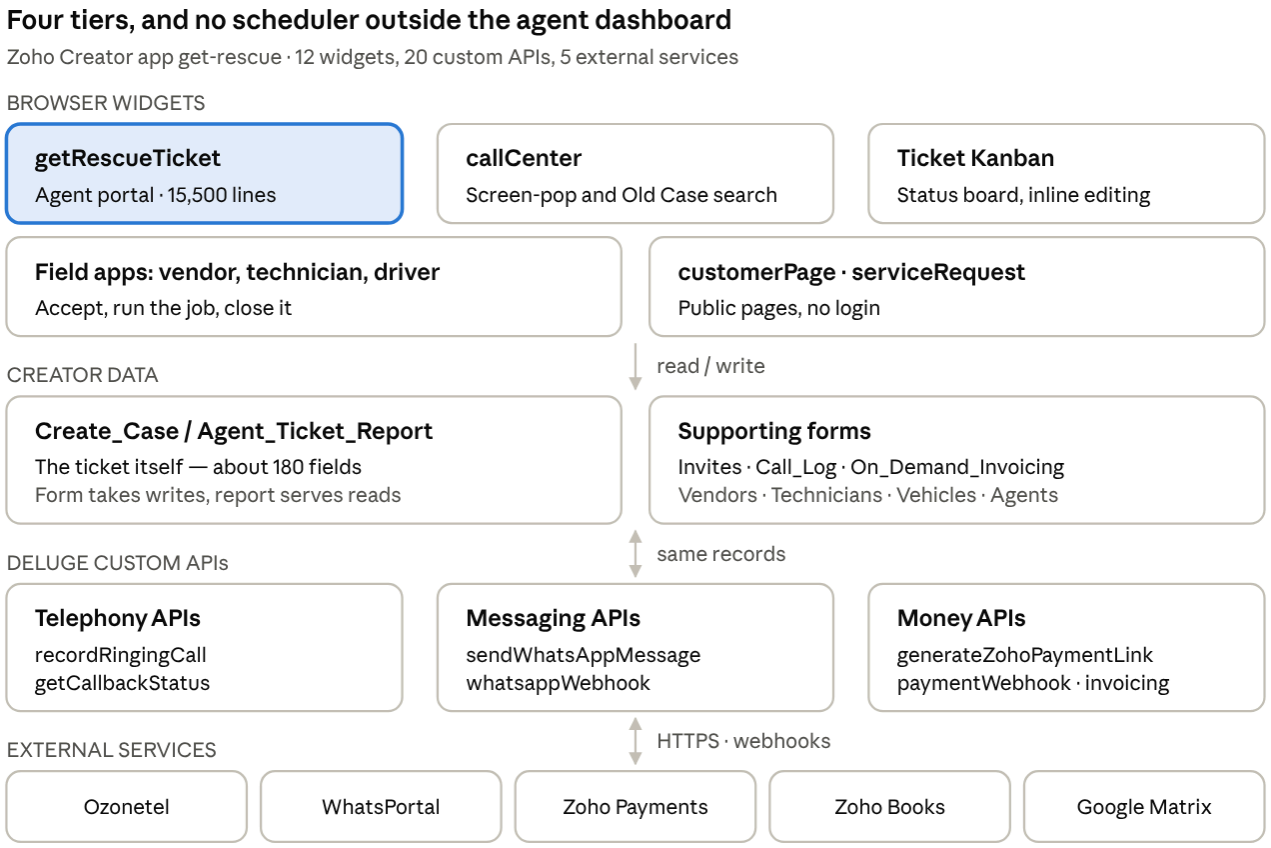
This document describes what is **in the deployed code**, read directly from the widget sources and the Deluge custom APIs, not from the requirement documents. Where the two differ, the code is what is recorded here and the difference is called out.

Two conventions are used throughout:

- **Built** means the code exists and has been read. It does not mean it has been tested against live Zoho data.
- **Needs verification** marks anything resting on a field name, report layout or third-party payload shape that has not been confirmed against the live system. Zoho fails silently when a field is missing from a report layout, so this distinction matters more here than in most projects.

Section 21 carries the full testing position. Nothing in this guide should be read as “confirmed working” unless that section says so.

# Platform architecture




platform architecture · 4 tiers

Widgets also invoke the custom APIs directly through Creator's JS SDK, so the middle two tiers are reached from above by both routes. Three further widgets are not drawn: `agentPerformance`, `vendorPerformance` and `toggleGetRescue`, all read-only or single-purpose.

The title states the constraint that matters most operationally. **There is no message queue, no scheduled Deluge function and no outbound webhook from Creator.** Everything time-based — the SEND 9 delay, scheduled-ticket promotion, picking up field-side changes — runs on the agent dashboard's 30-second poll, which only runs while somebody has that page open.

## Why widgets rather than native Creator pages

Creator's own forms could not deliver the staged wizard, the conditional field logic or the live distance calculations this workflow needs. The cost of that choice is that **every field name is a string in JavaScript**, resolved at runtime against a report. Nothing is type-checked, and a rename in Zoho breaks the widget silently — which is the root cause of most defects recorded in this guide.

### Deployment

Each widget is packaged with `zet pack` into a `.zip` and uploaded to Creator. Deluge files are pasted into their Custom API editors by hand. There is no CI, no automated deployment and no staging environment — changes go from a local edit to production in one step.

## Core data model

One form carries the whole job. `Create_Case` holds roughly 180 fields covering the customer, the vehicle, the money, four sets of coordinates, a dozen timestamps and the vendor assignment. Every other form exists to support it.

The app is `get-rescue` in the Zoho org `getrescued`, on the `.in` data centre. All API paths take the form `https://www.zohoapis.in/creator/v2.1/data/getrescued/get-rescue/`.

### Forms and reports

A Creator **form** takes writes; a **report** serves reads and updates. The widgets read reports and write forms, so a field present on a form but absent from the report layout reads back as `undefined` with no error. That trap has cost this project several defects.

| Form | Report | Holds |
|---|---|---|
| `Create_Case` | `Agent_Ticket_Report` | The ticket itself — everything below |
| `Invites` | `Invites_Report` | One row per vendor invited to a ticket |
| `Call_Log` | `Call_Log_Report` | One row per telephone call |
| `On_Demand_Invoicing` | `On_Demand_Invoicing_Report` | Zoho Books invoice requests |
| — | `Task_Rejections_Report` | Vendor and technician rejections |
| `Rate_Master` | `Rate_Master_Report` | Vendor rate cards. Report name is pattern-matched, **needs verification** |
| — | `Vendors_Report` | Vendor master, read at 1000 records |
| — | `Technicians_Report` | Technician master |

| Form | Report | Holds |
|---|---|---|
| — | `Vehicle_Master_Report` | Vehicle makes and models, with a `Category` of 2W or 4W |
| — | `Vehicle_Issue_Report` | Issue catalogue, filtered by `VEHICLE_TYPE` |
| — | `Client_Report` | B2B clients |
| — | `Agent_Report` | Agents, with `User_Type` driving permissions |
| — | `My_Availability_Agent` | The logged-in agent's own availability row |

## Field groups on Create_Case

### Identity and customer

`Case_ID` (the RSID label), `Customer_Name`, `Phone_Number1`, `Phone_Number`, `Alternate_Phone_Number`, `Email`, `Client`, `Client_Tracking_Id`, `Lead_Source`.

`Phone_Number1` is what the form shows; `Phone_Number` is what `Agent_Ticket_Report` actually searches on. The create stage mirrors one into the other. Both exist because they have carried the number at different times.

### Vehicle and issue

`Vehicle` (lookup to Vehicle_Master), `Vehicle_Registration_Number`, `Vehicle_Issue`, `Vehicle_Status`, `Service_Type`.

### Scheduling

`Time_of_service`, `Schedule_Date_Time`, `Computed_Service_Time`, `Added_Time`, `Confirmation_Time`.

### Money

`Total_Service_Fee`, `Booking_Fee`, `Remaining_Fee`, `Client_Charge`, `Rescue_Charge`, `Suggested_Rate`, `Rate_Breakup`, `Extra_Payable`, `Expense_Amount`, `Toll_Charges`.

Payment state: `Payment_Status`, `Payment_Method`, `Booking_Fee_Transaction_ID`, `Image_Upload` (the BF receipt), `Payment_received`, `Payment_Failure_Log`.

GST: `B2B_GST_Invoice_Required`, `GST_Number`, `GST_Name`, `GST_Address`.

### Four sets of coordinates

| Pair | Meaning |
|---|---|
| `Latitude` / `Longitude` | Breakdown location |
| `DropLocationLat` / `DropLocationLong` | Drop location, TOW only |
| `Reach_Location_Lat` / `Reach_Location_Lon` | Where the field party actually arrived |
| `Cancel_Location_Lat` / `Cancel_Location_Lon` | Where they were when they cancelled |

Plus `RSP_Start_Latitude` / `RSP_Start_Longitude` for the accept or drop position, and `Vendor_Fleet_Location_Lat` / `_Lon`.

**Distances** — all in kilometres

`Distance_To_Breakdown`, `Distance_To_Breakdown_TOW`, `Distance_Reach_To_Breakdown`, `Distance_Reach_To_Breakdown_TOW`, `Distance_Cancel_To_Breakdown`, `Cancel_Distance`, `Roundtrip_Distance`, `RT_KM`, `Distance_For_Quote_RT_KM`, `Office_To_Mechanic`, `Mechanic_To_Breakdown`, `Breakdown_To_Office`, `Breakdown_To_Drop_Distance`, `Drop_To_Office_Distance`, `Vendor_Distance`, `Vendor_Round_Trip_Distance`.

**Assignment**

`Vendors1` (the agent's multi-select pick list), `Assigned_Vendor`, `Assigned_Technician`, `Preferred_Vendor`, `Acceptance_Mode`, `Assignment_Time`.

**Timestamps**, in the order a job produces them

`Added_Time` → `Confirmation_Time` → `Assignment_Time` → `Service_Acceptance` or `Service_Acceptance_For_Tow` → `Reach_Time` → `Pickup_Time` (TOW) → `RSP_Completion_Time` → `Remaining_Fee_Receipt_Time` → `RSP_Closure_Time`. Off the happy path: `Service_Reject_Time`, `Cancellation_Time`.

**ETA**

`ETA`, `ETA_TOW`, `Agent_ETA_Override`, `Agent_ETA_Override_TOW`, `Return_Journey_ETA`.

**Evidence photos**

`Image_Upload` (BF receipt), `Image_Upload1` through `Image_Upload6`, `Pre_service_Photo`, `Expense_Photo`, `Upload_Photo_of_receipt`, `VCRF`.

The numbered names are historical and not self-describing; the labels in section 5 give each one its meaning.

### Naming conventions worth knowing

- A field prefixed `__` is **display-only**, built by the widget at render time and never saved — `__VendorMobileDisplay`, `__InvoicePdfDisplay`, `__CallHistoryDisplay`.
- A field prefixed `_` is **computed** for display — `_ExpectedArrival`, `_ETA_Met_Status`, `_BDL_ETA`.
- A field ending `_TOW` is the tow-branch twin of the RSR field of the same name. They are separate fields, not one field reused.
- `Case_ID` is set after creation as `RSID` plus the last six digits of the record id.

## Case lifecycle and the status model

Eighteen statuses live in `Create_Case.Status`. They are plain text, not a Creator picklist with enforced transitions, so the rules below are enforced by the widgets alone — anyone editing the record directly in Zoho can set any value.

### Every status

| Status | Written by | At what moment |
|---|---|---|
| *(blank)* | — | A record created but never saved past the first step |
| `SERVICE FEE QUOTE` | Agent wizard | Quote stage opened, fee not yet settled |
| `BOOKING FEE PENDING` | Agent wizard | A booking fee above zero has been quoted |
| `LOCATION REQUEST` | Agent wizard | Fee settled or waived; a location link has been sent |
| `LOCATION PENDING` | Agent wizard | Waiting on the customer's coordinates |
| `SCHEDULED` | Agent wizard | A future `Schedule_Date_Time` was set |
| `READY FOR ASSIGNMENT` | Agent wizard | Breakdown coordinates captured |
| `RSP IRA (INVITE RESPONSE AWAITED)` | Agent wizard | Assignment saved; invites created |
| `RSP ON THE WAY` | Vendor app | Vendor accepted the job |
| `WORK IN PROGRESS` | Vendor / technician | RSR: reached the breakdown |

| Status | Written by | At what moment |
|---|---|---|
| `LOADING` | Vendor / driver | TOW: reached the breakdown, loading the vehicle |
| `TO DROP LOCATION` | Driver app | TOW: vehicle picked up, in transit |
| `UNLOADING` | Driver app | TOW: arrived at the drop location |
| `REMAINING FEE DUE` | Vendor / technician / driver | Work complete, money outstanding |
| `RSP CLOSED` | Vendor app or agent | Final payment received |
| `CLOSED` | Agent closure | Agent completed the closure page |
| `RSP CANCELLED` | Vendor / technician / driver | Cancelled after acceptance |
| `RSP REJECT` | Vendor app | Invite rejected |
| `CX REJECT` | Agent | Customer refused the service |

`RSP CLOSED` and `CLOSED` both mean finished. Which one a ticket ends on depends on whether the vendor closed it from the field or the agent closed it from the closure page. Both are treated as closed everywhere that matters.

## The guard that stops statuses going backwards

The wizard re-runs every stage's `status:` callback on every save, which means a late save of an early step could otherwise drag a live job back to `LOCATION PENDING`. One shared predicate prevents it:

```
\text{statusStillEarly}(r, S) = \text{Status} \in \text{EARLY\_STATUSES}
\;\wedge\; \text{Status} \in S
```

In practice: a stage only writes its status when the ticket's current status is both still in the early set **and** one this stage is allowed to advance from. Otherwise it returns `r.Status` unchanged.

`EARLY_STATUSES` is the fixed list: blank, `SERVICE FEE QUOTE`, `BOOKING FEE PENDING`, `LOCATION REQUEST`, `LOCATION PENDING`, `SCHEDULED`, `READY FOR ASSIGNMENT`.

This is the single most load-bearing rule in the file. A defect in August 2026 — assigning a vendor then saving Accept in the same session corrupted the status — traced directly to it.

### The happy path, RSR

1. `SERVICE FEE QUOTE`
2. `BOOKING FEE PENDING` — skipped entirely when `Booking_Fee` is zero
3. `LOCATION REQUEST` → `LOCATION PENDING`
4. `READY FOR ASSIGNMENT`
5. `RSP IRA (INVITE RESPONSE AWAITED)`
6. `RSP ON THE WAY`
7. `WORK IN PROGRESS`
8. `REMAINING FEE DUE`
9. `RSP CLOSED` → `CLOSED`

### The happy path, TOW

Identical through step 6, then:

7. `LOADING`
8. `TO DROP LOCATION`
9. `UNLOADING`
10. `REMAINING FEE DUE`
11. `RSP CLOSED` → `CLOSED`

### Exits off the path

- **`RSP REJECT`** — a vendor declines the invite. The ticket returns to the assignment stage and can be re-invited; the rejecting vendor drops out of `Vendors1` on the next save.
- **`RSP CANCELLED`** — cancelled after acceptance, with `Cancel_Reason` (RSR) or `Reject_Reason2` (TOW) and a `Cancellation_Time`. Whether this is billable is **an open question with the client**; see section 20.
- **`CX REJECT`** — the customer refuses. `Rejection_Reason` carries why.

A ticket in `RSP CLOSED`, `CLOSED`, `RSP CANCELLED` or `CX REJECT` is treated as closed by the call-centre screen-pop, which will not offer it as a live case, and by the Kanban board, which opens it read-only.

## Agent portal: the ticket wizard

`getRescueTicket` is the main application — 15,500 lines, and the only place a ticket can be created. It presents `Create_Case` as a fourteen-stage wizard grouped into four buckets, with a dashboard in front of it.

## The fourteen stages

`branch` decides which stages a ticket sees: `all` for both service lines, `rsr` or `tow` for one, `pay` for the shared tail.

| # | Stage key | Title | Branch | Bucket | Save button |
|---|---|---|---|---|---|
| 1 | `create` | Create Service Ticket | all | Intake | Create Case |
| 2 | `quote` | Service Fee Quote & Booking Fee | all | Intake | Save & Continue |
| 3 | `locations` | Capture Locations | all | Intake | Save Locations |
| 4 | `assignment` | Vendor Assignment | all | Dispatch | Assign & Continue |
| 5 | `acceptance` | Service Acceptance | RSR | Dispatch | Save |
| 6 | `reach` | Reach Breakdown Location | RSR | On site | Save Progress |
| 7 | `wip` | Work In Progress | RSR | On site | Save |
| 8 | `acceptanceTow` | Service Acceptance (Tow) | TOW | Dispatch | Save |
| 9 | `reachTow` | Reach Breakdown (Tow) | TOW | On site | Save Progress |
| 10 | `wipLoadingTow` | Loading | TOW | On site | Save Loading |
| 11 | `wipReachedTow` | Reached Drop Location | TOW | On site | Save |
| 12 | `wipUnloadingTow` | Unloading & Handover | TOW | On site | Save Handover |
| 13 | `payment` | Final Payment Collection | pay | Settle | Save Payment |
| 14 | `agentClosure` | Agent Closure | pay | Settle | Close Case |

Stages 5–7 and 8–12 are mirror images. They write **different fields**, not the same ones — `Service_Acceptance` against `Service_Acceptance_For_Tow`, `ETA` against `ETA_TOW`, `Vendor_Email1` against `Vendor_Email2`.

## Resume: where a ticket reopens

Each stage declares which statuses resume on it. Reopening a ticket from the dashboard jumps straight to the right stage rather than restarting at step 1. `LOCATION REQUEST` and `LOCATION PENDING` both resume on `locations`; `READY FOR ASSIGNMENT` resumes on `assignment`.

## Stage 1 — Create Service Ticket

Customer, vehicle and issue. `Customer_Name`, `Phone_Number1`, `Alternate_Phone_Number`, `Email`, `Vehicle`, `Vehicle_Registration_Number`, `Client`, `Client_Tracking_Id`, `Vehicle_Issue`, `Vehicle_Status`, `Service_Type`, `Lead_Source`, `Time_of_service`, `Schedule_Date_Time`.

On save the record is created, then `Case_ID` is set to `RSID` + the last six digits of the new record id — a second write, because the id does not exist until the first one lands. The widget also mirrors `Phone_Number1` into `Phone_Number`, and stamps `Created_By_Agent_Email`.

The issue list is filtered by the vehicle's `Category` (2W or 4W) against `VEHICLE_TYPE` on the issue report.

## Stage 2 — Service Fee Quote & Booking Fee

The money stage. `Distance_For_Quote_RT_KM` and `Suggested_Rate` feed `Total_Service_Fee`, which splits into `Booking_Fee` and `Remaining_Fee`, with `Client_Charge` and `Rescue_Charge` for B2B work.

The arithmetic the stage keeps consistent:

```
\text{Client\_Charge} + \text{Rescue\_Charge} + \text{Booking\_Fee} =
\text{Total\_Service\_Fee}
```

Booking-fee collection fields: `Payment_Status`, `Payment_Method`, `Booking_Fee_Transaction_ID`, `Image_Upload`.

Three rules govern this stage, all of them client-reported fixes:

- `Booking_Fee` is **editable until `Payment_Status` is `PAID`**, then locked. Setting the status back to Pending unlocks it.
- The BF receipt photo is mandatory only when `Booking_Fee > 0` **and** a method is picked **and** that method is not `Payment Gateway`.
- `Booking_Fee_Transaction_ID` is hidden for `Cash` and shown for every other method.

A booking fee of zero skips `BOOKING FEE PENDING` entirely and goes straight to the location stage.

## Stage 3 — Capture Locations

Breakdown coordinates always; drop coordinates for TOW. Three ways in: a WhatsApp link the customer taps (`Send_Link_For_The_Breakdown_Location`), a lat/long the agent pastes (`LatLongPaste`), or an inbound WhatsApp location pin written by the webhook.

`Location_Received_Time` stamps when the coordinates arrived. `Calculate_Distance` fills `RT_KM`.

Saving only advances the status once real coordinates exist — a 2026-07-28 fix; before it the stage wrote `READY` whether or not anything had been captured.

### Stage 4 — Vendor Assignment

Covered in full in section 8.

### Stages 5–12 — the field stages

The agent can run these on the vendor's behalf. `Acceptance_Mode` records whether the vendor is using their own app or the agent is driving it manually, and several behaviours key off it — including whether the customer's ETA message gets a ten-minute pad.

Each stage captures its timestamp, its photo and its distance. `reach` writes `Reach_Time` and the reach coordinates; `wip` writes `Issue_Resolved`, before and after photos, and `RSP_Completion_Time`.

### Stage 13 — Final Payment Collection

`Remaining_Fee`, `Payment_Method_Final`, `Payment_received`, `Upload_Photo_of_receipt`, `Remaining_Fee_Receipt_Time`.

Once a remaining fee has been paid through the gateway specifically, the fee locks. Cash, UPI, card and net banking stay editable — a deliberate scope decision from 2026-08-19, taken from the client's own wording "online mode".

### Stage 14 — Agent Closure

Feedback, GST details, invoice generation, distance backfill. Covered in section 16.

### Cross-cutting behaviour

- **Ticket locking.** `Locked_By` and `Locked_At` stop two agents editing one ticket, with a heartbeat while the tab is open and a staleness timeout. **Needs verification** — both fields must be on the `Agent_Ticket_Report` layout or the lock silently does nothing.
- **File uploads** are deferred and run after the record save, so a failed upload never costs the agent their save.
- **Search** matches `Phone_Number` plus `Case_ID`, `Vehicle_Registration_Number` and `Client_Tracking_Id`, with `UCID` and `callerID` tried as an extended set that falls back if the report cannot query them.

## Dashboard, Kanban and analytics

Four surfaces show tickets in aggregate. They read the same `Agent_Ticket_Report` and differ only in how they slice it.

## The agent dashboard

Inside `getRescueTicket`, in front of the wizard. It pulls the most recent 1,000 tickets — Creator accepts only 200, 500 or 1000 for `max_records`, so this is the maximum.

It polls every 30 seconds while open, and that poll does more than refresh the list. Three background jobs hang off it:

| Job | What it does |
|---|---|
| `pollDashboardForMechanicUpdates` | Picks up field-side changes made in the vendor, technician and driver apps |
| `autoPromoteScheduledTickets` | Moves a `SCHEDULED` ticket into the live queue when its time arrives |
| `sendDueSend9Messages` | Sends the SEND 9 WhatsApp message five minutes after SEND 8 |

This matters operationally: **the dashboard is the application's only scheduler.** A Creator widget cannot hold a timer across a closed tab, so anything time-delayed happens only while somebody has the dashboard open. Section 15 covers the consequence for SEND 9.

Tickets carry live age timers and status chips. Search runs against phone number, `Case_ID`, registration number and client tracking ID.

## Ticket locking

Two agents opening one ticket is prevented by `Locked_By` and `Locked_At`, acquired on open and refreshed by a heartbeat. A lock older than **15 minutes** (`LOCK_STALE_MS`) is treated as abandoned and can be taken over, so a crashed browser does not strand a ticket.

**Needs verification:** both fields must be present on the `Agent_Ticket_Report` layout. If either is missing the read returns `undefined`, the lock appears to succeed, and nothing is actually held.

## The Kanban board

A separate widget, `Ticket Kanban`, showing tickets as cards in status columns. Each column declares the statuses it collects, and opening a card gives an inline editor for just that stage's fields rather than the whole wizard.

Columns map to the lifecycle: New → Quote (`SERVICE FEE QUOTE`, `LOCATION REQUEST`, `BOOKING FEE PENDING`) → Ready (`READY`, `LOCATION PENDING`) → Invite awaited → Acceptance → Work in progress → the TOW stages → settlement.

**Closed tickets open view-only** — added 2026-10-10. A ticket in a closed status shows a banner reading *"This ticket is already closed — view only"*, the field area stops accepting input, and the Save and Close buttons are hidden. Two guards sit on `saveStep` and `closeTicketAction` so a stale page cannot write through.

### Agent Performance

A small read-only widget over `Agent_Ticket_Report`, reporting per-agent ticket volumes and outcomes. No writes.

### Vendor Performance

The same shape for vendors — acceptance rates, rejections, completion. Also read-only, also sourced from `Agent_Ticket_Report` rather than a separate metrics store.

**Needs verification:** neither performance widget has been exercised against live data in any session recorded here. The metric definitions should be confirmed with the client before either is presented as a management report.

## Call centre and Ozonetel

Telephony runs on Ozonetel's CCaaS platform. This is the most actively changing area of the application and the one with open client questions, so it is documented in more detail than the rest.

### The three-step specification

The client set this out explicitly. It is the contract the implementation is measured against.

1. **Every incoming call creates a Call_Log row the moment it lands** — not at hangup.
2. **The RSID is attached at step 2**, when the agent presses Create Case or saves an assignment — never guessed from the phone number.
3. **Call details are written at the end of the call** — duration, status, recording.

Step 2 is the important one. Four separate places in the codebase used to guess a ticket from the caller's number, which attached new calls to old closed cases. All four were removed in October 2026.

## Call_Log fields

| Field | Written by | Meaning |
|---|---|---|
| `Phone_Number` | all creators | Caller, normalised to the last 10 digits |
| `UCID` | all creators | Ozonetel's call id |
| `Call_Status` | all | `RINGING`, `Incoming`, `Placed`, `Failed`, then Ozonetel's own DialStatus |
| `Call_Direction` | all | `Inbound` or `Outbound` |
| `Create_Case` | step 2 only | Lookup to the ticket |
| `Agent_Name` | Ozonetel | Their agent id |
| `Placed_By` | widgets | The logged-in Zoho email |
| `Call_Start_Time`, `Call_End_Time`, `Time_To_Answer`, `Call_Duration`, `Total_Duration` | end of call | All durations normalised to seconds |
| `Audio_File_URL`, `Conference_Audio_URL` | end of call | Recordings |
| `Call_Outcome`, `Transferred_To`, `Call_Flow` | end of call | Disposition and routing |
| `Call_Callback_History` | end of call | Append-only log, one timestamped entry per callback |

## Who creates a row

Three writers, which is the root of the duplicate problem below.

| Writer | When | Status stamped |
|---|---|---|
| `recordRingingCall.deluge` | Ozonetel ScreenPop fires at ring time | `RINGING` |
| callCenter screen-pop | The agent opens the pop | `Incoming` |

| Writer | When | Status stamped |
|---|---|---|
| `getCallbackStatus.deluge` | Ozonetel pushes the call summary | Ozonetel's DialStatus |

`recordRingingCall` is deployed but **not wired** — ScreenPop has not been configured to call it, so step 1 of the specification is not yet in force. Nothing in the code needs to change for it; it is an Ozonetel setting.

## The duplicate-record defect

Reported by the client on 10 October 2026: creating a ticket during a call produced two rows — one empty but linked, one complete but loose.

The cause: the screen-pop wrote a row carrying whichever id the ScreenPop URL supplied, and the end-of-call handler could not match it. Its exact-UCID match missed because the two ids differ, and its phone fallback was skipped because it only claims rows whose UCID is **blank**. So it created a second row.

Two fixes went in, in that order:

- **12 October, conditional:** the row is also matched on `monitorUCID` and on the raw URL parameters, not just `UCID`.
- **12 October, unconditional:** a row for that number still carrying `RINGING`, `Incoming` or `Placed` is claimed and completed in place. Those three words are written only by row creators and never by a callback, so such a row is by definition waiting for this call — whatever id it holds. This was the client's own suggestion.

The match chain is now five stages, tried in order:

1. Exact `UCID`
2. Alternate ids — `UCID`, `monitorUCID`, the `ucid` parameter, `callbackId`
3. Newest still-pending row for this number
4. Newest row for this number with a blank UCID
5. Create a row

Stage 3 is guarded so only the **newest** row for a number qualifies, which stops a stale `RINGING` row from an earlier lost callback being overwritten by an unrelated call.

**Needs verification.** The execution log names which stage matched, and that line answers an outstanding question — what identifier the ScreenPop URL actually carries — without needing Ozonetel support. If the log reads *"No existing Call_Log row matched. Creating one"*, the duplicate persists.

## Linking a call to a ticket

The client reported that a random older call was being attached. They offered two fixes and **Option 2 was built**: linking is restricted to the agent's own last call, if it is not already linked.

The rule, in `ensureCallLinkedToTicket` and `findLinkableCall`:

1. The UCID the page was opened with, if a row carries it.
2. A UCID stored on the ticket itself.
3. The agent's own most recent call row — attributed through `Placed_By` — **and only that one**. If it is already linked, nothing is linked; the search never walks further back.

That third rule is what removes the randomness: there is exactly one candidate, ever. It deliberately links **less** than before — a call the agent never popped has nothing attributing it to them, so it is left alone rather than guessed at.

`CALL_LINK_REQUIRE_OWN_CALL = false` restores the previous phone-and-recency search, which is kept whole behind the flag.

## The multiple-windows question

The client asked whether several open pop windows could leave several UCIDs active and link the wrong one.

**They cannot.** `POP_CALL_UCID` is a plain page variable read from that window's own URL. Nothing is stored in `sessionStorage` or `localStorage`, so windows cannot contaminate each other.

There is a related risk, though, and it is the one worth naming: the variable is deliberately never cleared, so a window opened for an earlier call still links **that** call if a ticket is created in it much later. Option 2 fixes exactly this case.

## The screen-pop

When a call lands, the pop shows the caller's number, any tickets carrying it, and an **Old Case** search box — the client's wireframe asked for search by mobile number, client tracking ID or vehicle registration, and Case ID was added alongside. The search reads only; the link happens when the ticket opens.

The pop is driven by a **poll**, not an iframe. Ozonetel's "Pop CRM" tab cannot frame a Zoho page — portal pages answer `X-Frame-Options: SAMEORIGIN` and public pages answer `DENY`, which is a Zoho-side header no URL or attribute changes. The working route is ScreenPop Server Side writing a row, and `pollForRingingCalls` noticing it.

### Other call APIs

| API | Purpose |
|---|---|
| `getInboundCallerInfo.deluge` | Caller lookup for the pop |
| `getVoiceCallbackDetails.deluge` | Pulls details for a placed call |
| `getCallbackStatusJson.deluge` | JSON wrapper, 11 lines |
| `cleanCallLogUcids.deluge` | One-off repair for 55 rows whose UCID was stored with a leading `:"` |
| `testCallLogReport.deluge` | Diagnostic |

Outbound dialling uses Ozonetel's `AgentManualDial`. It answers only `{"status":"queued successfully"}` and never returns a UCID — the id arrives later in the end-of-call push. An earlier version returned early when no UCID came back, which meant every successful call went unlogged.

## Vendor assignment

Assignment is a broadcast, not a hand-off. The agent picks several vendors, each gets an invite, and the first to accept takes the job.

### The vendor picker

Vendors come from `Vendors_Report`, read at 1,000 records. That cap matters: the default of 200 once meant any vendor past the 200th could not be resolved anywhere in the widget — blank in the picker, "(no vendor)" in accounting, and no rate card, so those cases could not be approved.

Ordering is **priority, then distance**, using `Vendor_Priority` and the straight-line distance from the breakdown.

The radius comes from `CONFIG.vendorDistanceKm`: **8 km for RSR, 20 km for TOW**. Vendors outside it are excluded from the default list.

A vendor's position is read from `Latitude` / `Longitude`, with `Current_Latitude` / `Current_Longitude` available for a live position. `Availability_Status` and `Vendor_Priority` come from the vendor master.

Setting `Preferred_Vendor` to Yes bypasses the radius filter and shows the full list.

## The Invites record

One row per vendor invited to a ticket.

| Field | Meaning |
|---|---|
| `RSID` | Lookup to the ticket |
| `Vendor` | Lookup to the vendor |
| `Invited_At` | When the invite was created |
| `Responded_At` | When the vendor answered |
| `Status` | `RSP REJECT` when declined |
| `Service_Acceptance_Next` | `Yes` when accepted |
| `Reject_Reason` | Why, on a rejection |

Acceptance and rejection are recorded in **different fields**, which is why reading the status takes a resolver rather than a single read.

## Invite status, as the Assign page computes it

| Shown | Condition |
|---|---|
| **Rejected** | `Status` is `RSP REJECT` |
| **Accepted** | `Service_Acceptance_Next` is `YES` |
| **Timed Out** | Still awaiting, and `Invited_At` is more than **2 minutes** old |
| **Awaiting response** | Everything else |

The two open states also show elapsed time. The timeout is `INVITE_TIMEOUT_MS`, set by the client on 3 October 2026 — it was a 15-minute placeholder before that.

Timed Out is **display-only**. Nothing is written back, so it cannot affect `Vendors1` or anything that saves.

## Re-invite

Offered only for a timed-out invite on a case nobody has taken. A rejected invite deliberately gets no Re-invite button: the vendor app keeps a local rejection stamp this write cannot clear, so the invite would look resent while staying hidden on the vendor's phone.

“Taken” is judged on two independent signals, either sufficient — an invite marked Accepted, or a non-blank `Assigned_Vendor`. Both are checked because the two have been written separately and this workspace has had spells where only one landed.

### Rejected vendors leave the pick list

`Vendors1` is the agent's own selection and nothing used to remove a vendor from it after they rejected, so re-saving the step resent the same rejected vendor. `pruneRejectedVendorsFromPick` now drops them. The full history, rejection reason included, stays in `Invites_Report`.

### Two defects worth recording

Both were live, both are fixed, and both illustrate traps specific to this platform.

**Criteria on a Lookup field.** `syncInvites` checked for existing invites with the server-side criteria `RSID == recordId`. Zoho rejects that outright — code 3330, *“Variable RSID does not exist in Invites”*. The failure was caught and treated as “no existing invites”, so **every re-save of the assignment step created duplicate invite rows**. It now fetches without criteria and filters client-side on the unwrapped id.

**An undefined function.** `lookupId()` was called in the Assign panel's renderer but never defined, throwing a `ReferenceError` exactly when there were invites to display — so the panel rendered blank precisely when it had something to show. This was the real cause of the client's *“vendor accepted but still shows pending”* report, after two earlier diagnoses that were both wrong.

### The empty-panel diagnostic

An empty panel used to render as nothing at all, which looks identical to “no vendors invited” and to a read that failed. It now states which it is, showing how many invite rows were read, how many matched, the ticket's record id, and whether `RSID` was among the fields the report returned.

That last check exists because a field missing from a report layout reads back as `undefined` with no error — the single most common silent failure in this application.

## Vendor app

`vendorTicket` is the vendor's own screen — 4,900 lines, used on a phone in the field. It is where a job is accepted, run and closed.

## Identity

The vendor is resolved from the logged-in portal user against `Vendors_Report`. This is harder than it sounds and has produced real defects.

`resolveVendorRecord()` matches on email. When it cannot prove which vendor is logged in, it still records a best guess in `MY_VENDOR_ID_UNPROVEN` rather than giving up, because an unproven id is still enough to find the right invite in most cases.

`findMyInviteByTicket(ticketId)` finds the vendor's own invite for a ticket. The original version compared `inv.Vendor.display_value`, which fails because **Creator returns a Lookup as a bare id, not an enriched object** — so every multi-vendor ticket failed to match. It now matches on record id first. This was the cause of the client's *"vendor 741 accepted but still shows pending"* report.

`unwrapLookup()` behaves differently between widgets — `getRescueTicket`'s is id-first, the three field apps' are display-value-first. `lookupId()` is id-first everywhere. Anyone editing these files needs to know which one they are calling.

## What the vendor does

| Action | Writes |
|---|---|
| Accept | `Service_Acceptance_Next: Yes` on the invite; `Status: RSP ON THE WAY` and `Assigned_Vendor` on the ticket |
| Reject | `Status: RSP REJECT` and `Reject_Reason` on both invite and ticket; `Service_Reject_Time` |
| Reached breakdown | `Reach_Time`, `Roundtrip_Distance`, `Status: WORK IN PROGRESS` (RSR) or `LOADING` (TOW) |
| Cancel | `Cancel_Reason` (RSR) or `Reject_Reason2` (TOW), `Status: RSP CANCELLED` |
| Reached drop | `Status: UNLOADING` |
| Dropped / handover | `Handover_to_Name`, `Handover_to_Number`, `Handover_to_Designation`, `Status: REMAINING FEE DUE` |
| Customer rejected | `Rejection_Reason`, `RSP_Completion_Time`, `Status: REMAINING FEE DUE` |

| Action | Writes |
| --- | --- |
| Payment received | `Payment_received: Yes`, `Payment_Status: PAID`, `Status: RSP CLOSED`, `Remaining_Fee_Receipt_Time`, `RSP_Closure_Time` |

A reject writes to the **invite and the ticket separately**. An earlier version wrote only one of them, which left the job looking live to the agent while the vendor saw it as closed.

### Responded_At

`stampInviteResponse()` writes `Responded_At` in a **separate update after the real one lands**, so a failure to write the timestamp can never cost the vendor their accept or reject. If the agent's RESPONDED column stays blank, the field is missing from the Invites form or its report layout — the console says exactly that.

### GPS capture

The vendor's position is captured at three moments and written to three different pairs of fields: `vendorReachGps()`, `vendorCancelGps()` and `vendorDropGps()`.

For a tow, the picked-up origin prefers the **stored reach position** over the live one — by the time the driver taps Picked Up they may have moved, and the breakdown-to-drop leg should measure from where the vehicle actually was.

### Customer ETA

The vendor app sends the ETA message itself. `ETA_AS_CLOCK_TIME = true` renders it as a 24-hour clock time rather than a duration, and `ETA_CUSTOMER_PAD_MINS = 10` adds ten minutes — but **only when the vendor ran the job in their own app**. An agent accepting on the vendor's behalf uses the agent's override with no pad.

### A platform note

`confirm()` returns `false` silently in mobile WebViews. Every confirmation in these apps is a built dialog, never the browser's.

---

## Technician and driver apps

`technicianTicket` (3,254 lines) and `driverTicket` (3,319 lines) are near-twins of the vendor app, scoped to one person and one branch.

### How they differ

| | Technician | Driver |
|---|---|---|
| Service line | RSR | TOW |
| Assigned via | `Assigned_Technician` | `Assigned_Technician` |
| Ends at | Issue resolved on site | Vehicle handed over at the drop |
| Extra actions | — | Reached drop, dropped, unloading |
| Photos | Pre-service, post-service | On-truck, additional |

The technician app has no loading, transit or handover steps. The driver app has no `Issue_Resolved` question.

### What they share with the vendor app

All three carry the same identity resolution, the same `findMyInviteByTicket`, the same `Responded_At` stamping, the same GPS capture and the same ETA logic. A change to one almost always needs making in all three — and the October 2026 rounds of fixes were applied to all three together.

### Technician assignment

A vendor can dispatch one of their own technicians. The vendor's technician list comes from `CONFIG.vendorTechniciansField` (`Technicians`) on the vendor record, and the chosen one goes onto `Assigned_Technician` with `Assigned_Technician_Email`.

**Needs verification:** the technician and driver apps have had less live exercise than the vendor app. Their invite-matching fix in particular has not been confirmed against a real multi-vendor ticket.

## Location, distance and ETA

The client sent a marked-up specification for this area on 7 October 2026 and scoped the Google API to four named calculations. That scope decision shapes everything below.

### Capturing coordinates

Four routes, in descending order of reliability:

1. **The customer taps a WhatsApp link.** `Send_Link_For_The_Breakdown_Location` sends a link to a Creator public page that posts the browser's geolocation back.

2. **The customer shares a WhatsApp location pin.** `whatsappWebhook.deluge` parses `location.latitude` / `location.longitude` and writes them — but **only into an empty breakdown location**. See section 15.
3. **The agent pastes a lat/long.** `LatLongPaste` and `DropLatLongPaste` accept a pasted pair.
4. **The field party's own GPS**, captured by the vendor, technician or driver app at reach, cancel and drop.

`Location_Received_Time` stamps the arrival. Reverse geocoding uses OpenStreetMap Nominatim, not Google — deliberately, to stay inside the client's scope.

## Two distance engines

| Engine | Used for | Cost |
|---|---|---|
| Haversine (`haversineKm`) | The vendor picker's radius filter, round-trip estimates | Free |
| Google Distance Matrix (`googleDistanceMatrix`) | The four billed calculations | Billed per call |

The picker uses straight-line distance on purpose: filtering 1,000 vendors through a billed API on every assignment would be the single most expensive thing the application does, and a radius filter does not need road accuracy.

The office origin is hard-coded:

```
\text{office} = (12.967945122837245,\; 77.6110507612277)
```

That is in central Bengaluru, which matches the service area — the public service-request form states *"Our Services are currently available only in Bengaluru."*

## Roundtrip_Distance

Written once, at Task Confirmation, by `writeRoundtripAtConfirmation()`. It is **never overwritten** — a re-save leaves the value alone.

| Service type | Legs measured |
|---|---|
| RSR | office → breakdown → office |
| TOW | office → breakdown → drop → office |

A TOW ticket confirmed before a drop location exists is left blank rather than written with a repair-shaped figure; Agent Closure backfills it.

### Vendor_Distance — the repair figure

Three branches, resolved at closure:

| Branch | Condition | Measures |
|---|---|---|
| 1 | Vendor reached the breakdown | Vendor start → reach position |
| 2 | Vendor cancelled | `Cancel_Distance` if already on file, otherwise vendor start → cancel point |
| 3 | No reach or cancel position | Vendor base → breakdown, from the agent's own entry |

### Vendor_Round_Trip_Distance — the tow figure

Four branches, branch 4 covering `Cancelled - Billable` and `Cancelled`.

**Open question:** whether plain `Cancelled` is the client's "Cancelled Not Billable" has never been answered. Branch 3 of `Vendor_Distance` and branch 4 here are both **billing decisions taken on an assumption**. They are listed in section 20.

### The named leg fields

`Office_To_Mechanic`, `Mechanic_To_Breakdown`, `Breakdown_To_Office`, `Breakdown_To_Drop_Distance`, `Drop_To_Office_Distance`.

**Open question:** whether `Office_To_Mechanic` and `Breakdown_To_Office` should be typed by the agent or calculated. Both are currently calculated.

### ETA

The ETA the agent sees and the ETA the customer receives are **not the same number**.

The agent's field ( `_ExpectedArrival` ) always prefers the override and formats as a 12-hour time. The customer's message is built by `customerEtaClock()`:

```
\text{ETA}_{\text{customer}} = \text{acceptance time} + \text{minutes} +
\text{pad}
```

| Route | Minutes used | Pad |
|---|---|---|
| Vendor ran the job in their own app | `ETA` or `ETA_TOW` | **+10 min** |
| Agent accepted on the vendor's behalf | `Agent_ETA_Override` or `_TOW` | none |

Rendered as a 24-hour clock time ( `ETA_AS_CLOCK_TIME = true` ). When the acceptance time cannot be parsed it falls back to the older duration text rather than sending something wrong.

The ten-minute pad is specification item 5.4. `_ETA_Met_Status` compares promise against `Reach_Time` to report ETA met or missed.

### Maps in the UI

Leaflet over OpenStreetMap tiles for display, with Google Maps links ( `https://www.google.com/maps/search/` ) for navigation. `Navigation_Link` and `Navigate_To_Breakdown_Link` hold deep links handed to the field party.

### What the client's spec asked for that is not built

- **Territory and pincode zones** — built on 10 October 2026, then reverted at the client's instruction the same day. No `Territory_Master` form, no `Breakdown_Pincode` field, no vendor `Territory` field. The geo-fence half (8 km / 20 km) has been in place since July and is unaffected.
- **Breakdown-to-drop origin** — the client said they would *"test and revert"*. No answer yet.

## Customer payments

Money is collected twice: a booking fee up front, and the remaining fee on completion. The same `Payment_Status` field is reused for both, which is a trap worth knowing about.

### The two collections

| | Booking fee | Remaining fee |
|---|---|---|
| When | Before dispatch | After the work |
| Amount | `Booking_Fee` | `Remaining_Fee` |
| Method | `Payment_Method` | `Payment_Method_Final` |

| | Booking fee | Remaining fee |
|---|---|---|
| Receipt photo | `Image_Upload` | `Upload_Photo_of_receipt` |
| Reference | `Booking_Fee_Transaction_ID` | — |
| Timestamp | `Booking_Fee_Receipt_Time` | `Remaining_Fee_Receipt_Time` |
| Collected by | Agent | Vendor, technician or driver |

`Payment_Status` carries `PAID` for **either** of them, confirmed live on 3 August 2026. So `PAID` alone cannot tell you which fee was paid. Anything needing to distinguish them checks `Payment_Method_Final` and `Payment_received` together — those two are only ever written by the final-payment action, in one atomic payload, so checking both is unambiguous.

## Payment methods

`Cash`, `UPI`, `Card`, `Net Banking`. **`Payment Gateway` is not currently an option on the field** — it needs adding in Zoho. The code already handles it correctly: a gateway payment needs no receipt photo and no transaction id.

## Zoho Payments

`generateZohoPaymentLink.deluge` builds a hosted payment link against `https://payments.zoho.in/api/v1`, independently of invoicing — no Books invoice need exist.

The reference id encodes which fee it is: `RSID645006-BF` for a booking fee, `-RF` for a remaining fee.

The `return_url` sends the customer somewhere useful rather than to a dead confirmation page — revised 22 September 2026 after client feedback on a live link:

| Context | Customer lands on |
|---|---|
| Booking fee | The customer page in breakdown-location mode, to share their location |
| Remaining fee | The customer page in feedback mode |

That is a deliberate sequencing choice: the moment after paying is the one moment the customer is definitely holding their phone.

Links are sent **on an agent click**, never automatically — `Send_Payment_Link` is a button, shown only when `Booking_Fee > 0`.

## The payment webhook

`paymentWebhook.deluge` receives the result. It is the cautionary tale of this project and is documented accordingly.

**The wrapper key is `event_object`, not `data`.** The first version was written against an assumed shape, and because a parameter merely *named* in a Deluge signature binds `null`, it failed silently for weeks with no payment ever reaching a ticket. `event_object` must be registered **with type Map** on the Custom API setup screen.

The real payload nests deeper than expected:

```
event_object
  └─ payment_links
       ├─ reference_id        e.g. RSID645006-BF
       ├─ meta_data[]         key/value pairs, including recordId
       └─ payments[]          each with payment_id
```

`payment_id` is **not** a direct child of the link — it sits inside `payments[]`.

## Failed payments

A `payment.failed` event appends to `Payment_Failure_Log` and **touches no payment field**. `Payment_Status` is deliberately left exactly as it was rather than forced to `PENDING`.

The reasoning is worth preserving: a customer who retries produces both a failed and a succeeded event, webhook delivery order is not guaranteed, and writing `PENDING` on the failure could overwrite a `PAID` that had already landed.

## A defect caused by retries

The `Payment_Method` default used to guess `Cash` when the field came back blank. On a webhook retry the field can be dropped, so a gateway payment would silently be relabelled as cash. The default no longer guesses on a fee already marked `PAID`.

## Extra charges

`Extra_Payable` and `Expense_Amount` with `Expense_Photo` cover costs arising during the job. `Toll_Charges` is a TOW-specific flag.

The customer is warned about extra charges by SEND 9, five minutes after the vendor's details arrive — see section 15.

# Vendor accounting

Built to the client's *"Zoho — Vendor Accounting"* specification, September 2026. Three levels: what the vendor earned, what Operations approves, and what Accounts pays.

## The payout calculation

$$\text{Final Payable} = \text{Vendor Net Fee} - \text{TDS} - \text{Cash Collected}$$

`Cash_Collected` is subtracted because a vendor who took cash from the customer has already been paid that much. **A negative Final Payable therefore means the vendor owes the company** — the cash they collected exceeded their fee.

Every figure is rounded to two decimals at each step, not once at the end.

## TDS

Per-vendor, from `TDS_Percentage` on the vendor record. Deducted **per batch, not per case**, then allocated back across the cases in the batch proportionally:

$$\text{batch TDS} = \text{round}\!\left(\frac{\sum \text{net fees} \times \text{rate}}{100},\, 2\right)$$

`allocateBatchTds` computes the batch figure first and `allocateProportional` distributes it, so the per-case amounts always sum exactly to the batch deduction. Rounding each case independently would not reconcile.

## Vendor GST

$$\text{GST Payable} = \text{Vendor Net Fee} \times 0.18$$

`VENDOR_GST_RATE = 18`, fixed. Unlike TDS this is **not per-vendor** — GST is set by the service category, not negotiated per supplier. 18% is the rate for roadside assistance, the same rate customer-side invoicing applies.

Applicability is read as *"this vendor has a GSTIN on file"* — `Vendor_GST_Number` being non-blank. No separate yes/no flag was added, on the reasoning that a GST-registered vendor is precisely the vendor who can charge it, and a second flag would need keeping in step with the number itself.

A blank GSTIN gives a GST Payable of **zero, not blank**, so the column always reconciles.

GST fields: `GST_Payable`, `GST_Amount`, `GST_Paid`, `GST_Paid_Date`, `GST_UTR_No`.

### Accounts Payable

The AP screen selects cases for one vendor and pays them as a batch. Rows can be **fee rows or GST-only rows**, and `apSelectionMode()` reports whether the ticked set is `fee`, `gst` or `mixed` — read from each checkbox's own `data-gstonly` rather than recomputed, so the mode can never disagree with what the row offered the user.

Payment is recorded by `recordVendorPayment.deluge` with `Vendor_Payment_Status`, `Vendor_Payment_Date` and a `UTR` number.

**Open question:** what should happen when a vendor's Final Payable is negative. At present it blocks the batch. Whether the company should net it against other cases, invoice the vendor, or carry it forward has not been decided with the client.

### Operations approval

An ops manager reviews the figures before they reach Accounts. Permission comes from `checkOperationsManagerPermission.deluge` and `User_Type` on `Agent_Report`, surfaced as `IS_OPS_MANAGER`.

`Ops_Approval_Comments` carries the reviewer's note.

**Needs action:** that field is currently a **Number** field in Zoho and must be changed to **Multi Line**. Until then a comment cannot be typed into it.

### Rate master

`Rate_Master_Report` holds vendor rate cards, loaded once at boot into `RATE_MASTER` and used to suggest a rate from distance and vehicle category.

**Needs verification:** the report name is pattern-matched from the form name, not confirmed live. If it is wrong, `RATE_MASTER` loads empty, rate suggestion silently stops working, and nothing errors.

### Not built

**The Books finance dashboard** — excluded by the client on 11 October 2026, along with Airtable sync. Vendor payment records exist in Creator but there is no consolidated finance view in Zoho Books.

## Invoicing and Zoho Books

Invoices are raised in Zoho Books through `onDemandInvoicing.deluge` against `https://www.zohoapis.in/books/v3`. It is the largest custom API in the project at 1,155 lines, and it takes 25 parameters.

## Tax configuration

Two hard-coded Books ids do the work, and both were established by live testing rather than from the documentation:

| Constant | Value | Why |
|---|---|---|
| `gst18_tax_id` | `3988332000000079033` | GST18 — **not** IGST18, which is an inter-state tax and wrong here |
| `gst_exemption_id` | `3988332000000091002` | Applied when `applyTax` is false |
| `gst_exemption_code` | `TAX` | Taken from a live response, not the public docs |

Books requires a line item to carry **either** a real `tax_id` **or** an explicit `tax_exemption_id`. Simply omitting the tax does not work — the item is rejected.

## Which invoice carries tax

An explicit client decision: **the booking-fee invoice carries no tax at all**. A reference number ending `-BF` is sent with `applyTax: "false"`, which routes it to the exemption id.

Customer-side GST is 18%, split as `cgst` and `sgst` — intra-state, which matches a Bengaluru-only service area.

## B2B GST details

Collected at Agent Closure: `B2B_GST_Invoice_Required`, `GST_Number`, `GST_Name`, `GST_Address`. A B2B customer needs the invoice raised against their own registration.

## When an invoice is raised

| Trigger | Status |
|---|---|
| `Generate_OnDemand_Invoice` button at closure | **Built.** Agent-initiated |
| Automatically on case closure | **Built** — added 14 September 2026 |
| At booking-fee collection | **Deliberately not called.** The client said *"don't use onDemandInvoicing for that — we will work on this later"* |

The third row is worth noting because the code for it exists and is commented out rather than deleted. If booking-fee invoicing is ever wanted, the call site is there.

### Reading the invoice back

`fetchInvoicePdf.deluge` retrieves the generated PDF. The closure page shows `__OnDemandInvoiceIdDisplay`, `__OnDemandInvoiceLinkDisplay` and `__InvoicePdfDisplay`, all display-only, with a `Fetch_Invoice_PDF` button.

Invoice links point at `https://books.zoho.in/invoices/`.

### The invoice URL in the WhatsApp message

Added 25 August 2026 at the client's request: the invoice URL is fetched and included in the closing WhatsApp message. When `onDemandInvoicing` returns no URL the message still goes — best-effort, so a Books failure never costs the customer their notification.

### Tracking

`On_Demand_Invoicing` and its report hold one row per invoice request, so a failed or duplicated raise can be traced.

**Needs verification:** the tax ids above are specific to one Zoho Books organisation. If the Books org changes, both ids change and every invoice will be raised with the wrong tax treatment — silently, because Books will accept them as valid ids for a different org's tax record only if they happen to exist there.

## WhatsApp messaging

WhatsApp is the only channel to the customer — they have no login and no portal. Everything they see arrives as a template message.

### The integration

Outbound goes through **WhatsPortal**, `https://api.whatsportal.io/v1/messages`, wrapped by `sendWhatsAppMessage.deluge`. Templates are pre-registered with parameter slots filled positionally.

Inbound arrives at `whatsappWebhook.deluge`, built 6 October 2026.

### The eleven templates

| Template | Sent when |
|---|---|
| `initiate_conversation` | Opens the conversation; carries a CONTINUE button |
| `confirm_booking_fee_and_total_fee` | The fee has been quoted |

| Template | Sent when |
|---|---|
| `note` | A free-text note alongside the fee |
| `tow_confirm_customer_location_details` | Requests the breakdown and drop locations |
| `inform_customer_request_confirmed_no_cancellation_after_5_mins` | The request is confirmed |
| `share_rescue_service_details_and_eta` | Vendor or mechanic name, number and ETA |
| `tell_customer_to_contact_the_call_center_before_agreeing_to_any_extra_work_or_extra_charges` | Five minutes after the details |
| `share_rescuer_reach_time_` | The rescuer has arrived |
| `mark_service_complete` | Work finished |
| `balance_payment_link` | Remaining fee is due |
| `google_feedback` | After closure, asking for a review |

## The SEND sequence

The client numbers these in the LOGIC-REPAIR and LOGIC-TOW specification sheets. The implementation follows that numbering.

| Step | Content | Stage |
|---|---|---|
| SEND 1–2 | Conversation opener and note | Booking fee page |
| SEND 3 | Booking fee and total fee | Booking fee page |
| SEND 4 | Location request | Location page |
| SEND 5–7 | Status milestones | On status change |
| SEND 8 | Vendor/mechanic name and number | On vendor acceptance |
| SEND 9 | Extra-charges warning | **Five minutes after SEND 8** |
| SEND 11 | Service complete | On completion |
| SEND 12 | Balance payment link | When the remaining fee is due |

SEND 10 does not appear in the code. **Needs verification** against the client's sheets — it may be a numbering gap in the specification rather than a missing message.

## SEND 9 and the five-minute delay

The specification says *"SEND 8 … and then send 9 after 5 minutes."* It went immediately from the moment it was wired, because **a widget cannot hold a five-minute timer** — the app is closed or the screen locked long before it would fire.

The delay was built on 10 October 2026 as a field plus a sweep, following the proven pattern of `autoPromoteScheduledTickets`:

1. On acceptance, `Send_9_Due_At` is stamped five minutes ahead.
2. The agent dashboard's 30-second poll picks up anything due.
3. **The due field is cleared before the message is sent**, so a second dashboard sweeping the same ticket reads a blank field and skips it.

Step 3 trades one failure mode for another, deliberately. If the send then fails the message is **lost rather than repeated** — for an advisory about extra charges, a duplicate is worse than a miss.

**This trades reliability for the gap, and the trade should be understood before it is accepted.** Today SEND 9 is delivered now, always. With the delay it is delivered in five minutes *if a sweeper is running*, and the only sweeper a Creator app has is a page somebody has open. Nobody on the dashboard all evening means SEND 9 does not go at all.

`SEND9_DELAY_MINUTES = 0` restores immediate sending, in one character.

**Needs action:** `Send_9_Due_At` (Single Line Text) must be created on `Create_Case`. Until it exists, the write is refused and all four accept paths fall back to sending immediately — which is exactly the old behaviour, so nothing breaks.

## Manual sends, by policy

Payment links and customer messages go out **on an agent click**, not automatically on reaching a step. This is a standing project rule, not an implementation shortcut. The one exception is the status-milestone sends, which fire on a status change.

## The inbound webhook

`whatsappWebhook.deluge` is the receiving half, and it is written to be deliberately tolerant because the exact payload shape WhatsPortal will send is **not yet known**. It reads three shapes:

| Shape | Example |
|---|---|
| Flat | `{"event":"…","from":"91…","text":{"body":"…"}}` |
| Wrapped | `{"data":{…}}` or `{"payload":{…}}` |
| Meta's own nesting | `{"entry":[{"changes":[{"value":{"messages":[…]}}]}]}` |

It logs the whole body regardless, so **one test delivery tells us the real shape** instead of another round of guessing. That design is a direct response to the `paymentWebhook` failure described in section 12.

Tickets are matched on the **last 10 digits** of the sender's number — WhatsApp reports `919876543210` while an agent may have typed `+91 98765 43210`. The newest matching ticket wins.

What it does with a message:

- **Text or button reply** → appended to `WhatsApp_Inbound_Log`, newest first, capped at 6,000 characters.
- **A location pin** → writes `Latitude` / `Longitude` and `Location_Received_Time`, but **only when the breakdown location is empty**. An agent may have corrected a pin the customer sent to the wrong place, and a TOW drop pin cannot be told apart from a breakdown one.
- **`Status` is never touched.** Capturing the location is what moves a ticket to READY FOR ASSIGNMENT, but that transition is computed by the wizard's own stage logic with its `statusStillEarly` guard. Writing a status from the webhook would race it.
- **Delivery and read receipts** are logged to the execution log and nowhere else. Every message produces sent, delivered and read callbacks; writing them onto the ticket would bury the customer's own replies under three machine lines each.

It always returns HTTP success, even on an error, because a non-2xx reply makes most providers retry the same delivery indefinitely.

## The CONTINUE gate — not built, on purpose

The specification wants a CONTINUE tap to release messages 2 and 3. The webhook **records the tap and reports it** but sends nothing.

Three reasons: it is a real behaviour change on the customer's phone, it contradicts the standing "agent clicks send" rule, and it cannot be tested until a real tap has arrived and been seen. The send hooks in at the `CONTINUE_TAPPED` branch once the client confirms

they want it automatic.

### Needed before inbound works at all

1. WhatsPortal must support outbound webhooks and be configured to call this URL. **That question is with Chinmay and is unresolved.**
2. Every parameter must be registered on the Custom API setup screen — `token`, `event`, `from` as String; `data`, `message`, `payload` as Map; `entry` as List. A parameter only named in the Deluge signature binds `null`, and the webhook returns `NOTHING_BOUND`.

### An open question on wording

Whether the `initiate_conversation` template body needs changing has not been established. The full registered wording needs reading before anyone answers that — it was not assumed here.

## Closure, feedback and reporting

The closure page is where the ticket is finalised, the money reconciled and the distances backfilled.

### Closure statuses

`Closure_Status` is separate from `Status`. It records the commercial outcome rather than the operational one.

| Value | Meaning |
|---|---|
| `Completed` | Work done |
| `Cancelled - Billable` | Cancelled, vendor still paid |
| `Cancelled` | Cancelled |
| `Not Converted` | Never became a job |
| `ON DEMAND` | Walk-in, outside the normal flow |

**Open question, and it is a billing one.** Whether plain `Cancelled` is the client's *"Cancelled Not Billable"* has never been answered. Both spellings appear in the codebase — `CANCELLED BILLABLE` and `CANCELLED NOT BILLABLE` in upper case alongside the mixed-case values above. The distance branches in section 11 depend on this, so the ambiguity has a money consequence.

### What closure does

1. Records `Closure_Status`.
2. Captures `Cx_Feedback_Score` and `Cx_Feedback_Text`.
3. Collects B2B GST details when `B2B_GST_Invoice_Required` is set.
4. Backfills distances the field apps could not compute — `Roundtrip_Distance` for a TOW confirmed without a drop location, and the `Vendor_Distance` branches.
5. Raises the Books invoice, automatically since 14 September 2026.
6. Offers `Google_Feedback_Requested`, which sends the `google_feedback` template pointing at `https://g.page/r/CdqrNPuv4uZ1EBM/review`.
7. Writes `Status: CLOSED`.

### Call history on the closure page

`__CallHistoryDisplay` shows the calls attached to this ticket. Since 9 October 2026 it shows **linked calls only** — a client decision.

Unlinked calls that share the ticket's number or UCID are **counted, not shown**, and the count is reported:

> No call is linked to this ticket. N unlinked call(s) share this ticket's number or call id — open the ticket in the wizard and use **Link Call** in the header to attach the right one.

That count exists so an empty table is never mistaken for a broken read. Before this change the page matched on phone number and UCID as well, which meant a repeat caller's unrelated calls appeared on every one of their tickets.

### Reporting and exports

A closed-ticket export is available from the dashboard, with the empty-state message *"No closed tickets to export yet"*. The Reject/Accept audit table, added 30 September 2026, lists invite responses across tickets and is gated to ops managers.

**Needs verification:** the export's column set and CSV escaping have not been confirmed against a real download in any session recorded here.

---

## Admin, users, roles and permissions

There is no admin module in the usual sense. Permissions are decided from the logged-in user's record and expressed as CSS visibility.

## How a role is decided

`LOGIN_EMAIL` comes from the Creator session. It is matched against `Agent_Report`, and `User_Type` on that record — a dropdown of `Agent` or `Operations Manager` — sets `IS_OPS_MANAGER`.

`checkOperationsManagerPermission.deluge` is the server-side counterpart, 17 lines.

**A production bug lives in this area's history.** When `Agent_Report` came back empty, `agentUserTypeFor()` could not resolve a type and `IS_OPS_MANAGER` silently became false, hiding ops features from a genuine ops manager. The function now warns explicitly when no record matches.

## What the ops-manager gate controls

Four sections, each hidden by default in CSS and revealed only for an ops manager:

| Class | Section | Added |
|---|---|---|
| `.opsmap` | Operations map | — |
| `.vendoracct` | Vendor Accounting, Level 1 — Operations Approval | 4 Sep 2026 |
| `.accountspayable` | Vendor Accounting, Level 2 — Accounts Payable | 4 Sep 2026 |
| `.inviteaudit` | Reject/Accept table | 30 Sep 2026 |

**This is a UI gate, not a security boundary.** The sections are hidden with CSS and the data is still fetched. Anyone able to open the widget's developer tools can reveal them. If vendor payout figures need to be genuinely restricted, that has to be enforced in Zoho's own permission model, not here.

## Portal users

Vendors, technicians and drivers log in through the Creator portal at `getrescued.zohocreatorportal.in`. Each app resolves the logged-in portal user against its own master report by email — see section 9 for why that resolution is fragile.

## Availability

`toggleGetRescue` is a 292-line widget doing one job: letting a vendor or agent set their own `Availability_Status`, read through `My_Availability_Agent`.

**Open question:** what Login Status is meant to mean, and whether it should gate assignment, has never been settled with the client.

### Customer-facing pages

Two public Creator pages, reached by permalink and needing no login:

| Page | Purpose |
|---|---|
| `Customer_Page` ( `customerPage` widget) | Location sharing, feedback, payment return landing |
| `serviceRequest` | Public service-request form — states that service is Bengaluru-only |

`inboundCallPop` (342 lines) is a small embedded page for the call pop, and `InboundCall` is its public page.

**Needs verification:** `customerPage` contains a *"Debug — Field Values"* block. Whether that is reachable by a customer on the live permalink has not been checked, and it should be — it is a customer-facing page.

## Additional modules not covered above

Three further widgets exist and are catalogued here for completeness.

| Widget | Lines | Purpose | Status |
|---|---|---|---|
| `agentPerformance` | 544 | Per-agent volumes and outcomes over `Agent_Ticket_Report` | Built, unverified |
| `vendorPerformance` | 562 | Per-vendor acceptance, rejection and completion rates | Built, unverified |
| `zoho_schema_import` | — | Schema import tooling, not a runtime module | Development aid |

### Supporting custom APIs

Beyond those already described:

| API | Lines | Purpose |
|---|---|---|
| `createTicket.deluge` | 344 | Server-side ticket creation, used by the public form |
| `searchTickets.deluge` | 106 | Server-side ticket search |
| `getVendorTickets.deluge` | 162 | The vendor app's own ticket list |

| API | Lines | Purpose |
|---|---|---|
| `getTaskRejections.deluge` | 116 | Rejection history |
| `getVehicleNames.deluge` | 124 | Vehicle master lookup |
| `cleanCallLogUcids.deluge` | 67 | One-off repair for malformed UCIDs |
| `testCallLogReport.deluge` | 102 | Diagnostic |

`cleanCallLogUcids` is worth a note: 55 Call_Log rows had stored their UCID as `:"93361501267909269`. The cause was order-dependent string stripping in `getCallbackStatus` — each cleanup pass ran once, in a fixed order, so a value arriving with two junk characters lost only the first. The parser now repeats the strip six times; the one-off API repaired the existing rows.

## End-to-end workflows

Six runs through the system, naming the field or function behind each step.

### 1. Inbound call to ticket

1. Customer rings the Ozonetel number.
2. ScreenPop fires `recordRingingCall.deluge`, which writes a `Call_Log` row with `Call_Status: RINGING`. **Not yet wired** — see section 7.
3. `pollForRingingCalls` in `callCenter` notices the row and opens the screen-pop.
4. The pop reads the caller's number, lists any tickets carrying it, and offers the **Old Case** search for mobile number, client tracking ID, registration number or Case ID.
5. The agent chooses: **New Case** opens `getRescueTicket` with the number and UCID pre-filled; **Old Case** opens the matched ticket.
6. On Create Case or on saving an assignment, `ensureCallLinkedToTicket` writes `Create_Case` onto the call row. **This is the only moment a call is linked.**
7. At hangup, Ozonetel pushes to `getCallbackStatus.deluge`, which completes the same row with duration, status and recording.

### 2. Intake to dispatch

1. Agent fills stage 1 and presses **Create Case**. The record is created; `Case_ID` is set to `RSID` + the last six digits of the record id.
2. Stage 2: the agent quotes `Total_Service_Fee` and splits it into `Booking_Fee` and `Remaining_Fee`. Status → `SERVICE FEE QUOTE`, then `BOOKING FEE PENDING` if the fee is above zero.

3. The agent clicks **Send Payment Link**. `generateZohoPaymentLink.deluge` builds a link with reference `RSID…-BF`; SEND 1–3 go to the customer.
4. The customer pays. `paymentWebhook.deluge` sets `Payment_Status: PAID` and stamps `Booking_Fee_Receipt_Time`. The `return_url` lands them on the customer page in location-sharing mode.
5. Stage 3: the location arrives — by the link, by a WhatsApp pin through `whatsappWebhook`, or pasted by the agent. `Latitude`, `Longitude` and `Location_Received_Time` are written.
6. Status → `READY FOR ASSIGNMENT`. `Confirmation_Time` is stamped, which triggers `writeRoundtripAtConfirmation()`.

A booking fee of zero skips steps 3 and 4 entirely.

## 3. Dispatch to acceptance

1. Stage 4: the picker lists vendors within 8 km (RSR) or 20 km (TOW), ordered by `Vendor_Priority` then straight-line distance.
2. The agent selects several into `Vendors1` and saves. Status → `RSP IRA (INVITE RESPONSE AWAITED)`.
3. `syncInvites` creates one `Invites` row per vendor with `Invited_At`.
4. Each vendor sees the job in `vendorTicket`.
5. First to accept wins: the invite gets `Service_Acceptance_Next: Yes` and `Responded_At`; the ticket gets `Assigned_Vendor`, `Service_Acceptance` (or `_For_Tow`) and `Status: RSP ON THE WAY`.
6. SEND 8 goes with the vendor's name and number. `scheduleSend9()` stamps `Send_9_Due_At` five minutes ahead.
7. A rejection writes `Status: RSP REJECT` and `Reject_Reason` to both invite and ticket, and the vendor drops out of `Vendors1` on the next save.
8. No response for **2 minutes** shows as **Timed Out**, with a Re-invite button if nobody has taken the case.

## 4. RSR job on the ground

1. The vendor dispatches a technician, or runs it themselves.
2. Travelling: `Distance_To_Breakdown` and `ETA` are set. The customer's ETA message is acceptance time + ETA + 10 minutes, as a 24-hour clock time.
3. Arrival: `Reach_Time`, the reach GPS into `Reach_Location_Lat` / `_Lon`, an arrival photo, and `Status: WORK IN PROGRESS`.

4. Work: pre- and post-service photos, `Issue_Resolved`, `Unresolved_Note` if not, `Expense_Amount` with photo if anything was spent.
5. Completion: `RSP_Completion_Time` and `Status: REMAINING FEE DUE`. SEND 11 goes.
6. Cancellation instead: `Cancel_Reason`, `Cancellation_Time`, the cancel GPS, and `Status: RSP CANCELLED`.

## 5. TOW job on the ground

Steps 1–2 as above, then:

3. Arrival: `Reach_Time`, `Odometer_reading_at_Reached_Location`, `Status: LOADING`.
4. Loading: on-truck photo, `VCRF`.
5. Picked up: `Pickup_Time`, `Status: TO DROP LOCATION`. The breakdown-to-drop leg measures from the **stored reach position**, not the live one.
6. At the drop: `Status: UNLOADING`.
7. Handover: `Handover_to_Name`, `Handover_to_Number`, `Handover_to_Designation`, `Status: REMAINING FEE DUE`.

## 6. Settlement

1. SEND 12 sends the `balance_payment_link` for `Remaining_Fee`.
2. The customer pays, or the field party collects cash. `Payment_received: Yes`, `Payment_Method_Final`, `Upload_Photo_of_receipt`, `Remaining_Fee_Receipt_Time`, `Status: RSP CLOSED`.
3. A gateway payment locks `Remaining_Fee` from further editing. Cash, UPI, card and net banking stay editable.
4. Agent Closure: `Closure_Status`, `Cx_Feedback_Score`, `Cx_Feedback_Text`, B2B GST details, distance backfill.
5. The Books invoice is raised automatically by `onDemandInvoicing.deluge`; its URL goes into the closing WhatsApp message.
6. `Google_Feedback_Requested` sends the review request. `Status: CLOSED`.
7. Ops review: `Vendor_Net_Fee` minus TDS minus `Cash_Collected` gives `Final_Payable`. `Ops_Approval_Comments` records the reviewer's note.
8. Accounts Payable pays a batch of cases for one vendor. TDS is computed per batch and allocated proportionally; `recordVendorPayment.deluge` writes `Vendor_Payment_Status`, `Vendor_Payment_Date` and the `UTR`.

### How data crosses module boundaries

| From | To | Carried by |
|---|---|---|
| callCenter | getRescueTicket | URL parameters `ticketId`, `ucid`, `ccPhone` |
| getRescueTicket | vendor app | `Invites` rows, keyed on `RSID` |
| vendor app | getRescueTicket | `Create_Case` fields, picked up by the dashboard poll |
| field apps | agent dashboard | `pollDashboardForMechanicUpdates`, every 30s |
| Zoho Payments | ticket | `paymentWebhook.deluge`, matched on `meta_data.recordId` |
| WhatsApp | ticket | `whatsappWebhook.deluge`, matched on the last 10 digits |
| Ozonetel | Call_Log | `getCallbackStatus.deluge`, matched through the five-stage chain |
| ticket | Zoho Books | `onDemandInvoicing.deluge`, 25 parameters |

Note what is **not** in that table: there is no message queue, no scheduled Deluge function and no webhook from Creator outward. Everything time-based happens on the agent dashboard's poll, which is why section 6 calls that poll the application's only scheduler.

## Business rules reference

Every constant, threshold and formula with its real name, so a change can be made in one place.

### Constants

| Name | Value | Where | Meaning |
|---|---|---|---|
| `vendorDistanceKm.RSR` | 8 km | `getRescueTicket` CONFIG | Vendor search radius, repair |
| `vendorDistanceKm.TOW` | 20 km | `getRescueTicket` CONFIG | Vendor search radius, tow |
| `INVITE_TIMEOUT_MS` | 2 min | `getRescueTicket` | Invite goes Timed Out. Client-set 3 Oct 2026 |

| Name | Value | Where | Meaning |
|---|---|---|---|
| `LOCK_STALE_MS` | 15 min | `getRescueTicket` | A ticket lock is abandoned |
| `CALL_AUTOLINK_WINDOW_MS` | 4 hours | `getRescueTicket` | How recent a call may be to auto-link |
| `SEND9_DELAY_MINUTES` | 5 | all four apps | Delay before SEND 9. `0` sends immediately |
| `ETA_CUSTOMER_PAD_MINS` | 10 | all four apps | Pad on the customer's ETA, vendor-app cases only |
| `ETA_AS_CLOCK_TIME` | `true` | all four apps | Clock time rather than a duration |
| `VENDOR_GST_RATE` | 18 | `getRescueTicket` | Vendor GST percentage |
| `CALL_LOG_PHONE_MATCH_MAX` | 20 | `getRescueTicket` | Cap on phone-matched calls shown |
| `CALL_LINK_REQUIRE_OWN_CALL` | `true` | `getRescueTicket` | Restrict linking to the agent's own last call |
| `fetchLatest` | 1000 | CONFIG | Tickets pulled for the dashboard |
| `OFFICE_LAT`, `OFFICE_LON` | 12.9679, 77.6111 | `getRescueTicket` | Office origin for distance legs |
| Poll interval | 20s / 30s | various | Dashboard and call-pop polls |
| Inbound log cap | 6,000 chars | `whatsappWebhook` | Conversation log truncation |

## Money formulas

**The fee split, which must always reconcile:**

```
\text{Client\_Charge} + \text{Rescue\_Charge} + \text{Booking\_Fee} =
\text{Total\_Service\_Fee}
```

**Vendor payout:**

$$\text{Final_Payable} = \text{Vendor_Net_Fee} - \text{TDS} - \text{Cash_Collected}$$

**Batch TDS, allocated proportionally afterwards:**

$$\text{batch TDS} = \text{round}\!\left(\frac{\sum \text{net fees} \times \text{TDS_Percentage}}{100},\, 2\right)$$

**Vendor GST, only when a GSTIN is on file:**

$$\text{GST_Payable} = \text{Vendor_Net_Fee} \times 0.18$$

**Customer ETA:**

$$\text{ETA}_{\text{customer}} = \text{acceptance time} + \text{minutes} + \begin{cases} 10 & \text{vendor app} \\ 0 & \text{agent on behalf} \end{cases}$$

Rounding is to two decimals at every step, not once at the end — otherwise the per-case TDS amounts do not sum to the batch deduction.

## Validation rules

| Rule | Condition |
|---|---|
| `Booking_Fee` editable | `Payment_Status` is not `PAID` |
| BF receipt photo mandatory | `Booking_Fee > 0` **and** a method is picked **and** it is not `Payment Gateway` |
| BF Transaction ID shown | A method is picked and it is not `Cash` |
| `Remaining_Fee` locked | Paid through the gateway specifically |
| Location stage advances | Real coordinates exist |
| Invite created | Only on an assignment save, deduplicated client-side |
| Call linked | Only at Create Case or assignment save |
| Status written | Only when `statusStillEarly` allows it |
| Pincode | 6 digits — **reverted, no longer present** |

## Deluge constraints that shape the code

These are not business rules but they explain why the Deluge files look the way they do. Every one was discovered by a rejected deployment.

- `{}` is a **map**, not an empty list. Use `List()`.
- A bare boolean variable cannot be an `if` condition — *"Comparison of non numeric expression is not supported"*. Compare a string flag instead.
- No `while` loop. Use a fixed list of passes.
- No arithmetic in a `for each` range bound.
- No method call on a literal — `"0123456789".contains(c)` is rejected.
- Long method chains off `ifnull()` are rejected; two methods is the proven safe limit.
- A `try` nested inside a `catch` is rejected.
- `for each` needs a variable, not an inline expression.
- Deluge reports the **wrong line number** for "Improper Statement", so the fix is always to find the bad *shape*, never to look where it points.

The proven-safe substitute for a list membership test is a delimited string with `contains()`, wrapped in pipes so a short value cannot match inside a longer one:

```
closedStatuses = "|CLOSED|RSP CLOSED|RSP CANCELLED|CX REJECT|";
statusKey = "|" + caseStatusText + "|";
if(closedStatuses.contains(statusKey)) { … }
```

## Zoho Creator constraints

- `max_records` accepts only **200, 500 or 1000**. Anything else is error 9250.
- A field on a **form** but missing from the **report layout** reads back `undefined` with no error. This has caused defects in `Added_Time`, `Invited_At`, `RSID`, `Responded_At`, `Locked_By` and `Locked_At`.
- Server-side criteria on a **Lookup** field are rejected — code 3330. Fetch without criteria and filter client-side.
- Lookup fields return as **bare ids**, not enriched objects, even with `field_config: "all"`.
- No Zoho page can be framed: portal pages answer `SAMEORIGIN`, public pages answer `DENY`.
- `confirm()` returns `false` silently in mobile WebViews.

# Requirements status

Separating what was originally scoped, what the client added later, what changed, and what is outstanding.

## Original scope, built

| Area | Notes |
|---|---|
| Ticket creation and the agent wizard | Fourteen stages, both service lines |
| RSR and TOW branching | Separate field sets throughout |
| Booking fee and remaining fee | Both collections, five methods |
| Location capture | Four routes in |
| Vendor assignment with multi-vendor invites | Priority then distance, geo-fenced |
| Vendor, technician and driver apps | Three separate widgets |
| Geo-fence radius 8 km / 20 km | In place since July 2026 |
| Status lifecycle | Eighteen statuses |
| WhatsApp notifications | Eleven templates |
| Zoho Payments | Links, webhook, return URLs |
| Zoho Books invoicing | Automatic on closure |
| Vendor accounting, three levels | Payout, ops approval, accounts payable |
| Kanban board | Status columns, inline editing |
| Ozonetel call logging | Screen-pop, logging, recordings |

## Client-added requirements

| Requirement | Date | Status |
|---|---|---|
| Invite timeout cut from 15 min to 2 min | 3 Oct 2026 | Built |
| Invoice URL in the closing WhatsApp message | 25 Aug 2026 | Built |
| Payment `return_url` to location or feedback | 22 Sep 2026 | Built |

| Requirement | Date | Status |
|---|---|---|
| Hide BF Transaction ID for cash | 23 Sep 2026 | Built |
| Remaining fee locked after gateway payment | 19 Aug 2026 | Built |
| Vendor accounting, Levels 1–2 | 4 Sep 2026 | Built |
| Reject/Accept audit table | 30 Sep 2026 | Built |
| Vendor GST from the vendor table | 26 Sep 2026 | Built |
| Ozonetel three-step specification | Oct 2026 | Steps 2–3 built; step 1 needs wiring |
| Closure page shows linked calls only | 9 Oct 2026 | Built |
| Old Case search in the pop | 10 Oct 2026 | Built |
| Closed tickets read-only in Kanban | 10 Oct 2026 | Built |
| Inbound WhatsApp location pin | 10 Oct 2026 | Built, blocked on WhatsPortal |
| SEND 9 after five minutes | 10 Oct 2026 | Built, needs a field |
| ETA as a clock time with a 10-min pad | Oct 2026 | Built |
| Duplicate call-log records | 12 Oct 2026 | Built, unverified |
| Link only the agent's own last call | 12 Oct 2026 | Built, unverified |

## Changes to existing features

| Change | Why |
|---|---|
| Four phone-guess RSID writes removed | They attached new calls to old closed cases |

| Change | Why |
|---|---|
| Closure call history narrowed to linked only | A repeat caller's unrelated calls appeared on every ticket |
| Invite existence check moved client-side | Server-side Lookup criteria are rejected, so every re-save created duplicates |
| `Payment_Method` no longer defaults to Cash on a PAID fee | A webhook retry relabelled gateway payments as cash |
| Rejected vendors pruned from `Vendors1` | Re-saving resent the same rejected vendor |
| `findMyInviteByTicket` matches on record id | It compared a display value against a bare id, so multi-vendor tickets never matched |
| `lookupId()` defined | It was called but never defined, blanking the Assign panel |
| UCID stripping repeated six times | Order-dependent stripping left `:"` on 55 rows |
| Vendor list raised from 200 to 1000 | Vendors past the 200th could not be resolved anywhere |

## Reverted

**Territory and pincode zones.** Built 10 October 2026 — a `Territory_Master` form, `Breakdown_Pincode` on the ticket, a vendor `Territory` field, and territory-aware ordering in the picker. Reverted the same day at the client's instruction. The geo-fence half is unaffected and still in force.

## Explicitly excluded by the client

Stated on 11 October 2026: *"right now we are not doing these point"*.

- Zoho Books finance dashboard
- Airtable sync
- REASSIGN button
- Structured QA and UAT
- Territory and pincode (added 11 Oct)

## Outstanding client decisions

None of these can be resolved by reading code. Each is a judgement the client owns, and several carry a money consequence.

| # | Question | Consequence |
|---|---|---|
| 1 | Is plain `Cancelled` the same as “Cancelled Not Billable”? | Vendor payout on cancelled jobs |
| 2 | `Vendor_Distance` branch 3 — measure from the vendor's base? | Billing |
| 3 | `Vendor_Round_Trip_Distance` branch 4 — same question for tows | Billing |
| 4 | What happens when Final Payable is negative? | Blocks a payment batch today |
| 5 | REASSIGN — appears on nine wireframe pages, never specified | Never built |
| 6 | Should the CONTINUE tap auto-send messages 2 and 3? | Customer experience |
| 7 | What does Login Status mean, and should it gate assignment? | Dispatch |
| 8 | Does the `initiate_conversation` template wording need changing? | Needs the registered text read first |
| 9 | `Office_To_Mechanic` / `Breakdown_To_Office` — typed or calculated? | Currently calculated |
| 10 | Breakdown-to-drop origin | Client said they would test and revert |
| 11 | Default payment method | Currently none — the agent must pick |
| 12 | Is SEND 10 a real message or a numbering gap? | One message may be missing |

## Zoho configuration still needed

| Setting | Blocks |
|---|---|
| ScreenPop → `recordRingingCall` | Ring-time call rows |
| `Send_9_Due_At`, Single Line Text on `Create_Case` | The SEND 9 delay |
| `whatsappWebhook` parameters registered with correct types | All inbound WhatsApp |
| `Payment Gateway` added to `Payment_Method` | Recording gateway payments |

| Setting | Blocks |
| --- | --- |
| `Ops_Approval_Comments` Number → Multi Line | Typing an approval comment |
| `Locked_By` / `Locked_At` on `Agent_Ticket_Report` layout | Ticket locking |

## Testing status and known limitations

This section is the most important one for anyone taking the project over, and it is the least comfortable.

### The honest position

**Almost nothing in this application has been through structured testing.** There is no test suite, no UAT record and no regression pack. Structured QA and UAT were explicitly excluded by the client on 11 October 2026.

What exists instead: syntax and brace-balance checks run before each deployment, and defects found in live use and reported by the client. That is how every fix listed in section 20 was discovered.

The distinction this guide draws throughout:

| Label | Means |
| --- | --- |
| **Built** | The code exists and has been read |
| **Verified** | It has been exercised against live Zoho data and behaved correctly |

Most of the application is Built. Relatively little is Verified.

### Verified by live use

These have been exercised in production, mostly because something went wrong first and was then confirmed fixed:

- Ticket creation, the wizard stages, status transitions
- Booking-fee and remaining-fee collection, including the gateway
- Vendor assignment and multi-vendor invites
- WhatsApp outbound templates
- Ozonetel call logging and recordings

- Zoho Books invoice generation
- Payment webhook — after the `event_object` discovery

## Built but not verified

| Area | Why it is unverified |
|---|---|
| Duplicate call-log fix | Deployed 12 Oct 2026, no test call made yet |
| Agent's-own-last-call linking | Same |
| Inbound WhatsApp, including location pins | Blocked: WhatsPortal outbound webhook support is unconfirmed |
| SEND 9 five-minute delay | Needs `Send_9_Due_At` created first |
| Ticket locking | Needs both fields on the report layout |
| Agent and Vendor Performance widgets | Never exercised against live data in any recorded session |
| Closed-ticket export | Column set and CSV escaping unconfirmed |
| `Rate_Master_Report` name | Pattern-matched, never confirmed |
| Technician and driver invite matching | Fixed alongside the vendor app, never tested on a real multi-vendor ticket |
| Vendor accounting batch payments | Calculations read correct; a real batch has not been traced end to end |

## Known limitations

**The dashboard is the only scheduler.** Anything time-delayed — SEND 9, scheduled-ticket promotion — runs only while somebody has the agent dashboard open. Nobody on the dashboard means nothing fires. This is a platform constraint, not a bug, but it needs to be understood operationally.

**Role gating is cosmetic.** Ops-manager sections are hidden with CSS and their data is still fetched. Developer tools reveal them. If vendor payout figures must be genuinely restricted, that needs enforcing in Zoho's permission model.

**Silent failures are the norm on this platform.** A field missing from a report layout reads `undefined` with no error. A wrong report name loads an empty list with no error. A Deluge parameter not registered binds `null` with no error. Several weeks of broken payment handling came from exactly this. Any change touching a field or report name should be verified against live data, not assumed.

**`Payment_Status` is shared between two collections.** `PAID` cannot tell you which fee was paid. Code distinguishing them must check `Payment_Method_Final` and `Payment_received` together.

**`unwrapLookup()` differs between widgets.** `getRescueTicket`'s is id-first; the three field apps' are display-value-first. `lookupId()` is id-first everywhere. Copying code between widgets without checking this has already caused one defect.

**Three near-duplicate field apps.** `vendorTicket`, `technicianTicket` and `driverTicket` share most of their logic by copy rather than by a module. Every fix must be applied three times, and the October 2026 rounds were.

**The `getRescueTicket` widget is 15,500 lines in one file.** It carries extensive inline commentary recording why each decision was made, which is valuable, but the file is at the limit of what is comfortable to navigate.

## Deployment and verification practice

| Check | Tool |
|---|---|
| JavaScript parses | A script-block syntax checker, run before each pack |
| Deluge braces balance | A comment-and-string-aware brace counter |
| Widget packaging | `zet pack`, run per widget immediately after editing |
| Change isolation | `git diff` reviewed hunk by hunk before packing |

One caution recorded from experience: the first brace counter skipped strings but **not comments**, so an apostrophe in a `//` comment opened a phantom string and reported balanced files as broken. Verify a checker against an unmodified file before trusting its verdict.

## What should be tested first

In priority order, because each has a client report behind it:

1. One call log row per call, not two — and read the `getCallbackStatus` execution log to see which match stage fired.
2. Linking takes the agent's own last call only.
3. Vendor accept and reject showing in the Assign page.
4. Closure page showing linked calls only.
5. TOW booking fee editable, BF receipt conditional.

A run sheet covering these and six further areas exists separately from this document.

## Glossary

| Term | Meaning |
|---|---|
| **AP** | Accounts Payable — Level 2 of vendor accounting |
| **BF** | Booking Fee — the upfront payment, before dispatch |
| **CCaaS** | Contact Centre as a Service — Ozonetel's platform |
| **Creator** | Zoho Creator, the low-code platform this runs on |
| **CX** | Customer. `CX REJECT` means the customer refused the service |
| **Deluge** | Zoho's server-side scripting language |
| **Form** | A Creator object that accepts writes |
| **IRA** | Invite Response Awaited — invites sent, nobody has answered |
| **Lookup** | A Creator field referencing another record. Returns a bare id |
| **RF** | Remaining Fee — collected after the work |
| **Report** | A Creator view that serves reads and updates. **A field on the form but not on the report layout reads back `undefined`** |
| **RSID** | The human-readable case label: `RSID` + the last six digits of the record id |
| **RSP** | Roadside Provider — the vendor. Most statuses are prefixed with it |
| **RSR** | The on-site repair service line |
| **SEND n** | The client's numbering for the WhatsApp message sequence |
| **TDS** | Tax Deducted at Source — withheld from vendor payouts, per vendor |
| **TOW** | The recovery-to-a-drop-location service line |

| Term | Meaning |
|---|---|
| **UCID** | Ozonetel's unique call id |
| **UTR** | Unique Transaction Reference — the bank reference for a vendor payment |
| **VCRF** | Vehicle Condition Report Form — a TOW evidence field |
| **WhatsPortal** | The WhatsApp Business API provider |
| **Widget** | An HTML/JS application embedded in a Creator page |
| **zet** | Zoho's widget packaging CLI |

## Field-name conventions

| Prefix or suffix | Meaning |
|---|---|
| `__Something` | Display-only, built at render time, never saved |
| `_Something` | Computed for display |
| `Something_TOW` | The tow-branch twin of the RSR field of the same name — a **separate field** |
| `Image_Upload1` – `6` | Historical photo field names, not self-describing |

## Reading this codebase

The widget sources carry dated inline comments recording why each decision was made, often including the client's own words and the defect that prompted the change. Those comments are the most reliable history the project has — more so than the commit messages, which are mostly *"bug fixes"*. Anyone changing this code should read the comment above the line before changing the line.